

Using Data Flow Diagrams

CSE 4407

Md. Bakhtiar Hasan

Assistant Professor
Department of Computer Science and Engineering
Islamic University of Technology

July 25, 2025



Why Visualize Data?

- **Challenge:** Understanding how data **moves** and **transforms** within an organization based *only* on interviews or documents can be complex and confusing
 - **Think:** Trying to assemble complex furniture with only text instructions!
- **Solution**
 - Visual models help!
 - Crystallize information for both analysts and users
- **Enter DFDs:** A way to graphically represent data processes and flow

Introducing Data Flow Diagrams (DFDs)

- What

- A Structured Analysis Technique
- Creates a graphical representation of data processes and flows
- Uses just **four** basic symbols
- Useful in both Agile and traditional SDLC approaches

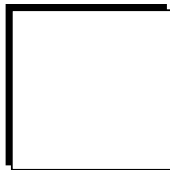
- Why

- Freedom
 - Avoids early commitment to specific technology
 - Focus on *what* the system does, not *how*
- **Understanding:** Clarifies how systems and subsystems interact
- **Communication:** A common visual language for users and analysts
- **Analysis:** Helps verify if all necessary data and processes are defined for a proposed system

DFD Symbols: External Entity and Data Flow

- External Entity

- **Meaning:** A source or destination of data, *outside* the system being analyzed
- **Naming:** Use a Noun (e.g., Customer, Accounts Payable Dept, Inventory System)
- **Note:** Can appear multiple times on a DFD to avoid crossed lines



- Data Flow

- **Meaning:** Movement of data between Entities, Processes, and Data Stores
- **Naming:** Use a Noun describing the data (e.g., New Customer Order, Payment Details, Validated Address, Student Record)
- **Tip:** Parallel arrows can show simultaneous flow



DFD Symbols: Process

- Process

- Meaning

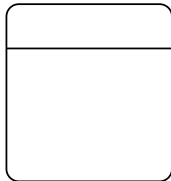
- Transforms incoming data flow(s) into outgoing data flow(s)
 - Represents work being done

- **Key:** Data flow *leaving* a process **MUST BE** different from the data flow *entering* it

- **Naming:** Crucial for clarity!

- **System Level:** Name of the whole system (e.g., Order Processing System)
 - **Subsystem Level:** Name of a major part (e.g., Payment Verification Subsystem)
 - **Detailed Level:** Verb-Adjective-Noun (e.g., Validate Customer Credit, Calculate Order Total, Generate Shipping Label)

- **Numbered:** Indicates layers



DFD Symbols: Data Store

- Data Store

- Meaning

- A repository where data is held
 - Allows storage, examination, retrieval

- Naming

- Use a Noun describing the data held (e.g., Customers, Products, Orders, Student Records)
 - Plural Nouns often work well

- **Numbered:** Labeled with 'D' and a unique number

- **Logical vs. Physical**

- Logical DFDs *do not* specify the medium
 - Physical design shows 'how'

- **Note:** Temporary files or transient data are usually *not* shown as data stores



The Big Picture: Top-Down Approach

- **Systematic Development:** Don't just start drawing randomly!
- **Analogy:** Like using Google Maps - Start with the whole region, then zoom into cities, then streets
- **Key Idea:** Layering/Decomposition
 - Start general → Context Diagram
 - Add detail layer by layer (Diagram 0, Child Diagrams)
- **Process Summary**
 - List business activities → Identify Entities, Flows, Processes, Stores
 - Draw Context Diagram (Overall system view)
 - Draw Diagram 0 (Explode Context Diagram; show major processes/stores)
 - Draw Child Diagrams (Explode Diagram 0 process for more detail)
 - Check for errors and refine
 - (Later Steps: Develop Physical DFD, Partition for implementation)

Step 1: The Context Diagram

- Provides an overview or “context” of the system
- Purpose
 - Shows the system boundaries, major inputs/outputs, and external entities
 - Simple, uncluttered view of the system’s scope
- Characteristics
 - Contains ONLY ONE process symbol → Represents the *entire* system
 - This process is numbered “0”
 - Shows ALL external entities interacting with the system
 - Shows major data flows between entities and the system
 - NO data stores are shown at this level

Step 2: Exploding Diagram 0

- **Decomposition:** Exploding the Context Diagram's single process (Process 0)
- **Purpose:** Shows the major **processes** within the system and how they connect via **data flows** and **data stores**
- **Characteristics**
 - Represents the *entire* system, but broken down
 - Includes 3 to 9 major processes
 - Processes are numbered sequentially (1, 2, ..., N)
 - Shows major data stores used by these processes
 - Shows the same external entities and boundary data flows as the Context Diagram → Inputs/outputs must match!

Visualizing Decomposition: Context vs. Diagram 0

Context-Level DFD:

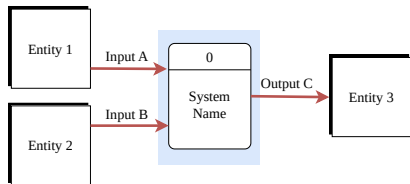
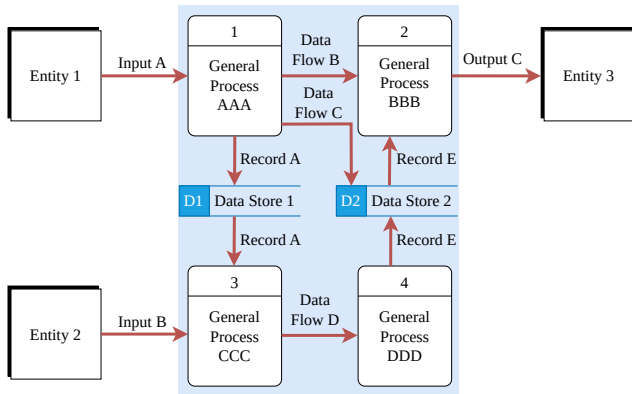


Diagram 0:



Drawing Strategies

- **Input-Focused**
 - Start with an external entity providing input
 - Ask: “What happens to this data next?”
 - Follow it through processes and stores
- **Output-Focused**
 - Start with a system output (report, screen)
 - Ask: “Where did the data for this output come from?”
 - Work backward to find sources
- **Data-Store Focused**
 - Pick a data store
 - Ask: “What processes write to this store? What processes read from it?”
- **Process-Focused**
 - Pick a key process
 - Ask: “What data does it need? What data does it produce?”
 - Connect them

Step 3: Creating Child Diagrams (Deeper Dive)

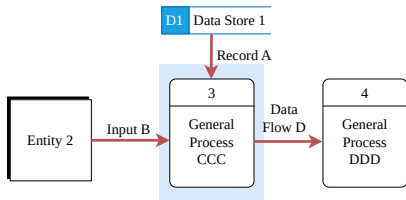
- **Further Decomposition:** Any process on Diagram 0 (or other non-primitive diagrams) can be exploded into its own Child Diagram
- **Terminology**
 - **Parent Process:** The process being exploded
 - **Child Diagram:** The resulting detailed diagram
- **Purpose:** To show the internal details of a specific parent process
- **Numbering Convention:** Child processes use the parent's number, a decimal, and a unique suffix
- **Functional Primitive**
 - A process that is NOT exploded further
 - Simple enough to be described without another DFD

Child Diagrams: Balancing is Key!

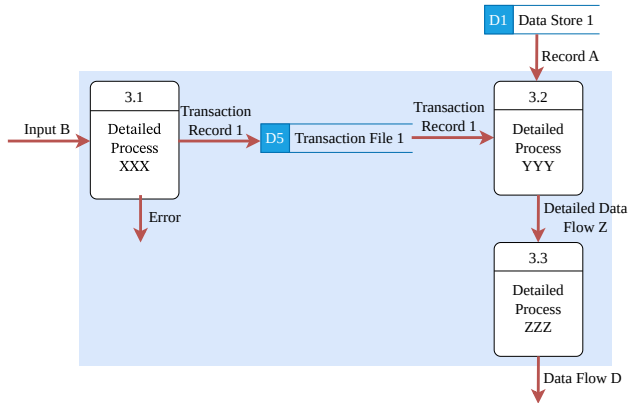
- **The Golden Rule:** Vertical Balancing
 - Inputs/Outputs of the Child Diagram **MUST** match the Inputs/Outputs of its Parent Process
 - **Interface Data Flows:** Flows entering/leaving the Child Diagram boundary correspond to flows entering/leaving the Parent Process symbol
- **What is new then?**
 - **New Local Data Stores:** Stores used only *within* the child processes (e.g., temporary lookup tables)
 - **New Internal Data Flows:** Flows *between* processes within the Child Diagram
 - **Minor Error/Exceptional Flows:** Flows not crucial enough to show on the higher-level parent diagram

Visualizing Decomposition: Diagram 0 vs. Child

Diagram 0:



Child:



Core Transformation Rules

- Existence of Process
 - Every DFD must include ≥ 1 process
 - No freestanding entities, stores, or flows
 - **Common mistake:** Entire diagram shows only entities and stores
- Process I/O
 - Each process needs ≥ 1 incoming data flow and ≥ 1 outgoing flow
 - Processes represent transformation, not data sources/sinks
 - **Common mistake:** Process that never outputs or no inputs
- Connectivity
 - Every data store must connect to ≥ 1 process
 - Stores cannot speak directly to entities or other stores
 - **Common mistakes**
 - Isolated stores
 - Entity $\leftrightarrow$ Store, Store $\leftrightarrow$ Store, Entity $\leftrightarrow$ Entity

Labeling and Decomposition Rules

- Naming Convention

- **Process:** Verb-Adjective-Noun (e.g., Verify Customer Credit)
- **Data Flows:** Descriptive noun phrase (e.g., Credit Application)
- **Common Mistake:** Vague labels like “Process Data” or “Info”

- Decomposition

- Child diagrams must match parent process I/O → Vertical balancing
- ≤ 9 processes per diagram for readability
- **Common Mistakes**
 - Parent shows two inputs but child diagrams show three
 - Too many processes crammed into Diagram 0

Two Flavors of DFDs: Logical vs. Physical

- DFDs differ based on the perspective they represent
- Logical DFD
 - Focuses on the **business logic**
 - Describes **WHAT** the business does, the events that occur, and the data involved
 - Independent of *how* it's implemented (technology, people)
- Physical DFD
 - Focuses on the **system implementation**
 - Describes **HOW** the system will be (or is) constructed
 - Includes details like hardware, software, files, specific people/roles
- **Analogy:** Logical → Architect's sketch vs. Physical → Wall materials, plumbing routes, electrical wiring, etc.

Logical DFDs: Focusing on the Business (“What”)

- **Purpose:** Model the business operations and data flow conceptually
- **Based on:** User narratives, business events
- **Characteristics:** Depicts how business operates
 - **Processes:** Represent business activities (e.g., “Approve Credit”, “Fulfill Order”)
 - **Data Stores**
 - Represent collections of data needed, regardless of storage method (e.g., “Customer Information”, “Product Details”)
 - Shows *permanent* data collections
 - **Controls:** Show business rules/controls (e.g., “Credit limit must be checked”)
- **Advantages**
 - **Better communication** with users (uses business terms)
 - More **stable** systems (based on business needs, not fleeting tech)
 - Better **understanding** of the business by analysts
 - **Flexibility** and easier maintenance
 - Helps **eliminate redundancies**

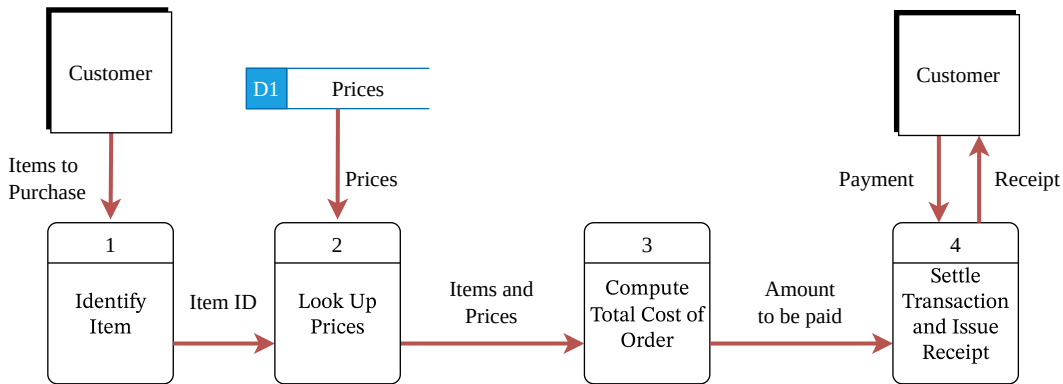
Physical DFDs: Focusing on Implementation (“How”)

- **Purpose:** Model how the system will be technically implemented
- **Based on:** Logical DFD, existing system details, technology choices
- **Characteristics:** Depicts how the system will be implemented
 - **Processes:** Represent specific programs, modules, or manual procedures (e.g., “Run Program VALIDATE.EXE”, “Manually Stamp Invoice Paid”)
 - **Data Stores**
 - Represent actual physical files, database tables, manual files (e.g., “CUST_MASTER.DBF”, “TEMP_ORDER_FILE.TXT”, “Archived Invoice Binder”)
 - Includes master *and* temporary/transaction files
 - **Controls:** Show system/data validation controls (e.g., “Verify Record Found Status”, “Check Batch Totals”, “Log Security Events”)
- **Advantages**
 - Clarifies **manual vs. automated** processes
 - Describes processes in **more detail**
 - Shows **sequencing** needs
 - Identifies **temporary data stores**
 - Specifies **actual file/report names**
 - Adds necessary **system controls**

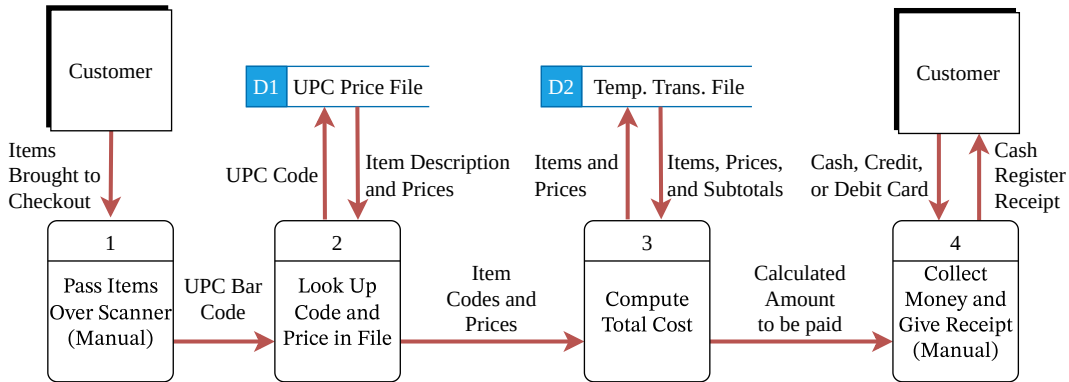
The Typical Development Flow

- Ideal Progression
 - Analyze Current System: Understand how things work *now*
 - Derive Current Logical DFD
 - Extract the underlying business logic from the current implementation
 - What business purpose does the current system serve?
 - Develop Proposed Logical DFD
 - Design the ideal business flow for the new system → Add/remove/change features based on requirements, using the current logical as a base
 - What should the new system do?
 - Develop Proposed Physical DFD
 - Plan the implementation of the new logical design
 - How will the new system be built?
- Rationale: Understanding the core business logic first (logical) provides a stable foundation before deciding on implementation details (physical)

Logical DFD Example: Grocery Checkout



Physical DFD Example: Grocery Checkout



Physical DFD Details: Stores and Data

- Intermediate Data Stores

- Common in Physical DFDs
- Hold data temporarily between processes that run at different times
- **Analogy:** Grocery Shopping
 - Shopping Cart (Links Selecting Items and Checking Out)
 - Shopping Bags (Links Checking Out and Transporting Home)
 - Fridge/Cupboards (Links Transporting Home and Preparing Meal)
 - Plate/Bowl (Links Preparing Meal and Eating Meal)

- Base vs. Derived Elements

- Base Element

- Data that is keyed in/captured directly
- Must be stored
- **Example:** Quantity Ordered, Customer Address

- Derived Element

- Data that is calculated or results from logic
- Doesn't necessarily need to be stored → Can be recalculated
- **Example:** Order Total, Sales Tax Amount

Tracking Data Operations: The CRUD Matrix

- **Acronym:** **C**reate, **R**ead, **U**ppdate, **D**eleate
- **CRUD Matrix:** A table showing which processes perform which CRUD actions on which major data stores/entities
- **Purpose**
 - Helps ensure all necessary data handling actions are accounted for in the system design
 - Useful check during physical DFD development

Activity	Customer	Item	Order	Order Detail
Customer Logon	R			
Item Inquiry		R		
Item Selection		R	C	C
Order Checkout	U	U	U	R
Add Account	C			
Add Item		C		
Close Customer Account	D			
Remove Obsolete Item		D		
Change Customer Order	RU	RU	RU	CRUD

Building DFDs from Events

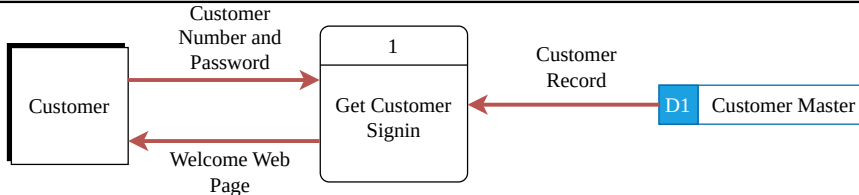
- **Event Modeling:** A practical approach, especially for physical DFDs
- **Event:** Something that happens that triggers the system to respond

Event	Source	Trigger	Activity	Response	Destination
Customer logs on	Customer	Customer number and password	Find customer record and verify password. Send Welcome web page.	Welcome web page	Customer
Customer browses items at Web storefront	Customer	Item information	Find item price and quantity available. Send Item Response web page.	Item Response web page	Customer
Customer places item into shopping basket at Web storefront	Customer	Item purchase (item number and quantity)	Store data on Order Detail Record. Calculate shipping cost using shipping tables. Update customer total. Update item quantity on hand.	Item Purchased web page	Customer

Event Modeling Example: DFD Fragments 1

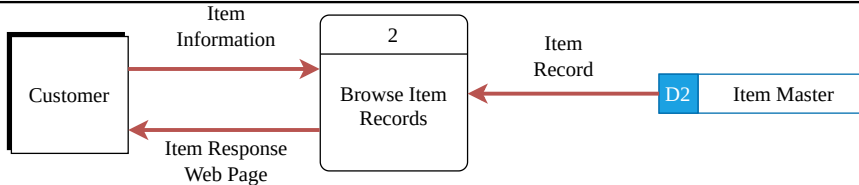
- **Table Rows to DFDs:** Each row in the Event Response Table can be mapped directly to a DFD “fragment” → Often a single process with its I/O
- **Combining Fragments:** These fragments are then assembled to build up the larger DFD → Often starting with Diagram 0

Event	Source	Trigger	Activity	Response	Destination
Customer logs on	Customer	Customer number and password	Find customer record and verify password. Send Welcome web page.	Welcome web page	Customer



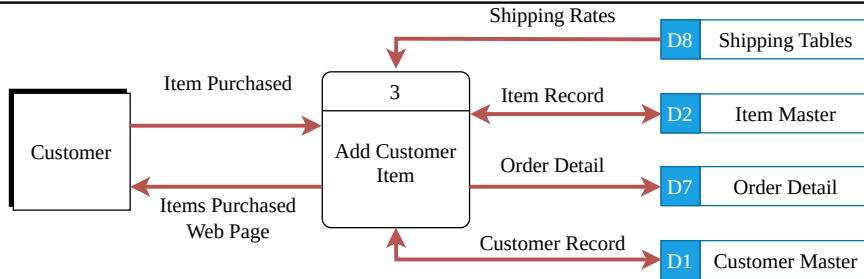
Event Modeling Example: DFD Fragments 2

Event	Source	Trigger	Activity	Response	Destination
Customer browses items at Web storefront	Customer	Item information	Find item price and quantity available. Send Item Response web page.	Item Response web page	Customer



Event Modeling Example: DFD Fragments 3

Event	Source	Trigger	Activity	Response	Destination
Customer places item into shopping basket at Web storefront	Customer	Item purchase (item number and quantity)	Store data on Order Detail Record. Calculate shipping cost using shipping tables. Update customer total. Update item quantity on hand.	Item Purchased web page	Customer



Linking DFDs with Use Cases

- **Use Cases**
 - Describe interactions between an actor (user) and the system to achieve a goal
 - Summarizes an event
- **Connection:** Each Use Case often corresponds directly to one (or more) DFD processes
- **Use Case Details Help Define Processes**
 - Trigger → Input Data Flow
 - Steps → Process Logic (potentially sub-processes on a child DFD)
 - Inputs/Outputs → Data Flows and Data Stores needed

Demo: [Use Case - Add Customer Item](#)

Partitioning DFDs: Preparing for Implementation

- **What:** Dividing a DFD (usually Physical) into logical units for implementation
 - Manual Procedures
 - Computer Programs/Applications/Modules
- **How:** Often shown by drawing dashed lines around grouped processes on the DFD
- **Why**
 - **Different User Groups:** Separate programs for different departments/locations/roles
 - **Timing:** Processes running at different times must be in separate programs (e.g., daily update vs monthly report) → Consider web page partitioning for long interactions
 - **Similar Tasks:** Group related functions together in one module
 - **Efficiency:** Combine processes that use the same large files to reduce I/O
 - **Consistency:** Run related reports/updates together to ensure data snapshots are consistent
 - **Security:** Separate sensitive processes (e.g., login, payment) into secure modules/programs → Separate secure vs. non-secure web pages

The Case: World's Trend

- **Who:** World's Trend - Mail order supplier of fashion clothing
- **Steps**
 1. **Understand the business activities**
 - **Goal:** The raw material for our DFDs
 - **Outcome:** Helps identify entities, flows, processes, stores
 2. **Context Diagram** → The Big Picture
 - **Goal:** Show the overall system boundary and major interactions
 - **Derived from:** The activity list → Identify external players and key inputs/outputs
 - **Reminder:** Only ONE process (0), all entities, major flows, NO data stores
 3. **Diagram 0** → Major Processes and Stores
 - **Goal:** Explode Process 0 to show major internal factors
 - **Derived from:** Deeper analysis of the activity list → Identify key steps and data repository
 - **Note:** Processes are general, numbering applied, labels are key

The Case: World's Trend

- Steps

- 4. Logical Child Diagram

- Goal: Explode a process from Diagram 0 to show its internal logic
 - Note: Subprocess numbering, more detailed logic, no external entities shown directly

- 5. Physical Child Diagram

- Goal: Explode a process from Diagram 0 showing *how* it will be implemented
 - Contrast with Logical: Focus on sequence, specific actions, real file/report names, temporary files needed for implementation

- 6. Partitioning the Physical DFD

- Goal: Group physical DFD processes into implementation units (programs, procedures)
 - Based On: Reasons like timing, user groups, efficiency, security
 - Result: Suggests how the system could be broken down into separate programs/jobs

Why Partition Websites?

- **Breaking Down Complexity**
 - Websites often involve multi-step interactions
 - Partitioning helps manage this
- **Key Benefits**
 - **Usability**
 - Simpler, less cluttered web forms/pages per step
 - Easier for users to understand and complete tasks
 - **Speed**
 - Faster page loads
 - Processing can be quicker if broken into logical steps
 - **Maintenance**
 - Modular design makes updates easier
 - Smaller, focused programs are simpler to manage and debug

When to Partition vs. When to Use Ajax?

- **Decision Point:** How fine-grained should partitioning be?
- **Partition when**
 - Need to get data from a **new data store** or an **external partner** (e.g., checking inventory, validating credit card)
 - Fetching a **large amount of data** from the server (e.g., displaying search results like available flights)
 - Crossing a **security boundary** (e.g., moving to payment processing requiring HTTPS)
 - Guiding users through a **distinct sequence of complex inputs**
- **Ajax (Asynchronous JavaScript and XML) when**
 - Need to update **small parts** of the current page without a full reload (e.g., refreshing a stock ticker, validating a single field, dynamically populating a dropdown list based on a previous selection)
 - Avoids creating too many tiny pages for minor interactions

Example: Online Airline Booking

- **Scenario:** Designing the airline booking part of a web-based travel site
- **DFD Shows Partitioning:** Each process is often treated as a separate partition, implying distinct web interactions or back-end calls
- **Overall Flow**
 - Customer provides Dates/Airports (P1)
 - System displays available flights (P2) → D1 Flight, creates D2 Flight Details
 - Customer selects flight (P3) → Interacts with Airline for price/availability, uses D2
 - Customer selects seat (P4) → Logic implied, uses P3 output
 - Customer books flight (P5) → Uses D3 Customer Master, creates D5 Flight Reservation
 - Payment processed (P6) → Interacts with Credit Card System
 - Notifications sent (P7, P8) → To Airline and E-ticket to Customer

Demo: [Airline Reservation](#)

Analyzing the Partition (Travel Site - 1)

- **Finding Flights:** P1 Select Criteria, P2 Display Flights
 - P2 needs to search internal data store (D1 Flight) → Significant data operation
 - Results (D2 Flight Details) need to be stored temporarily to display potentially multiple pages of flights without slow re-searchers
 - Separates user input from data retrieval/display
- **Selecting Flight/Seats:** P3 Select Flight, P4 Select Seats
 - P3 needs real-time price/availability from the external Airline system
 - P4 often involves dynamic, interactive display based choice → A distinct user interaction step

Analyzing the Partition (Travel Site - 2)

- **Booking and Payment:** P5 Book Flight, P6 Charge Credit Card
 - P6 involves interaction with external Credit Card System
 - Payment processing requires a secure connection (HTTPS), often handled separately from other booking steps → Security boundary dictates partitioning
- **Notifications:** P7 Update Airline Flights, P8 Produce E-Ticket
 - P7 involves sending data to external Airline
 - P8 is a distinct final output task triggered after successful booking/payment
- **User Input/Form Complexity:** Breaking the booking into multiple steps results in simpler, less intimidating forms for each step → Improves usability

More Than Just Diagrams

- **Early Stages:** Requirements Gathering
 - Higher-level DFDs (Context Diagram, maybe Diagram 0) provide a *visual overview*
 - Complements narrative information (interviews, stories)
 - Helps analysts and users grasp the big picture of data movement quickly
- **Later Stages:** Detailed logical and physical DFDs help in design specification and implementation planning

The Power of Labels

- Avoid Generic Labels

- **Bad:** Input, Process Data, Output, File, Information
- **Why**
 - They convey almost no specific meaning
 - Every system has input, process, output!

- Use Meaningful, Specific Labels

- **Good:** Customer Order, Validate Credit Status, Approved Order, D1 Customer Master, Shipping Confirmation Email
- **Why:** They clearly state what data is flowing or what action is happening

- **Goal:** Make the diagram understandable to users, team members, and future analysts *without* needing the original creator to explain it

Lasting Documentation

- **Built to Last:** Assume your DFDs will outlive your involvement in the project
 - Especially true for consultants, but relevant for everyone
 - Future analysts, maintainers, or developers will rely on them
- **Purpose**
 - Document system logic at various levels
 - Substantiate the data flows and transformations within the organization's systems
- Clarity and Accuracy are paramount